DBMS · INTERVIEW NOTES

Database Management *Systems* Notes

Definitions, ER & relational models, normalization, SQL, indexing, transactions and recovery — with formulas and solved numericals. Everything an SDE interview probes.

16	60+	9	6
SECTIONS	DEFINITIONS	SOLVED QS	NORMAL FORMS

CONTENTS

01 Introduction to DBMS
02 Database Architecture & Data Independence
03 The ER Model
04 Relational Model & Keys
05 Functional Dependencies
06 Normalization
07 SQL & Joins
08 Indexing
09 Transactions & ACID
10 Concurrency Control
11 Deadlocks in DBMS
12 Recovery
13 Storage & File Organization
14 SQL vs NoSQL
15 Practice Numericals (Solved)
16 Quick Revision Cheat-Sheet
17 Follow-up Drills

01 Introduction to DBMS

DEFINITION

A **Database Management System (DBMS)** is software that stores, retrieves, manages and controls access to a **database** — an organised collection of related data. It sits between the user and the physical data.

DBMS vs File System

	FILE SYSTEM	DBMS
Redundancy	High (data duplicated)	Controlled (normalisation)
Consistency	Hard to maintain	Enforced by constraints
Concurrency	Poor	Handled via transactions/locks
Security	Limited	Role-based access control
Querying	Manual code	SQL (declarative)

Advantages of a DBMS

- Reduced data redundancy and inconsistency; enforced integrity.
- Concurrent access with **ACID** guarantees and crash recovery.
- Data security, backup/restore, and independent access via SQL.

02 Database Architecture & Data Independence

DEFINITION — 3-SCHEMA ARCHITECTURE

The **three-schema architecture** separates a database into three levels: the **Internal** (physical storage), the **Conceptual** (logical structure), and the **External** (user views), so changes at one level need not affect the others.

LEVEL	DESCRIBES	SEEN BY
External	Individual user views	End users / apps
Conceptual	Whole logical schema (tables, relationships)	DBA / designers
Internal	How data is physically stored (files, indexes)	System

DATA INDEPENDENCE

Logical independence = change the conceptual schema without changing external views.
Physical independence = change storage without changing the conceptual schema.

03 The ER Model

DEFINITION

The **Entity-Relationship (ER) model** is a high-level, graphical data model that describes data as **entities** (objects), their **attributes** (properties) and the **relationships** between them.

CONCEPT	MEANING
Entity	A real-world object (Student, Course).
Attribute	A property (name, roll_no). Types: simple, composite, derived, multivalued.
Relationship	An association between entities (Student *enrols* Course).
Cardinality	1:1, 1:N, N:1, M:N.
Weak entity	An entity that cannot exist without its owner (dependent).

ER → RELATIONAL

Each entity → a table; each attribute → a column; an **M:N** relationship → a separate table with the keys of both entities.

04 Relational Model & Keys

DEFINITION

The **relational model** represents data as **relations (tables)** of **tuples (rows)** and **attributes (columns)**. It is the foundation of SQL databases.

KEY	DEFINITION
Super key	Any set of attributes that uniquely identifies a row.
Candidate key	A *minimal* super key (no unnecessary attributes).
Primary key	The candidate key chosen to identify rows; cannot be NULL.
Alternate key	A candidate key not chosen as primary.
Foreign key	An attribute referring to the primary key of another table (referential integrity).
Composite key	A key made of two or more attributes.

05 Functional Dependencies

DEFINITION

A **functional dependency** `X → Y` means that the value of attribute set X **uniquely determines** the value of attribute set Y — for any two rows with the same X, Y must also be the same.

Trivial	Y ⊆ X (always holds).
Partial	A non-prime attribute depends on *part* of a candidate key.
Transitive	X → Y and Y → Z imply X → Z (Y non-prime).
Full	Y depends on the whole of X, not a subset.

ARMSTRONG'S AXIOMS

```
Reflexivity: if Y ⊆ X then X → Y
Augmentation: if X → Y then XZ → YZ
Transitivity: if X → Y and Y → Z then X → Z
```

Attribute closure X^+ = the set of all attributes functionally determined by X. Used to find keys and check dependencies.

06 Normalization

DEFINITION

Normalization is the process of organising data into tables to reduce **redundancy** and eliminate **anomalies** (insert, update, delete), by decomposing relations based on functional dependencies.

FORM	CONDITION (REMOVES)
1NF	No multi-valued / composite attributes — all values atomic.
2NF	1NF + no **partial** dependency (non-prime → part of a key).
3NF	2NF + no **transitive** dependency (non-prime → non-prime).
BCNF	For every FD X → Y, X is a **super key**.
4NF	BCNF + no multi-valued dependency.

EXAMPLE

R(Student, Course, Instructor) where Course → Instructor and (Student, Course) is the key. `Course → Instructor` is a partial dependency → violates 2NF. Split into (Student, Course) and (Course, Instructor).

TRADE-OFF

Normalization reduces redundancy but increases the number of joins. High-read systems often **denormalize** for speed.

07 SQL & Joins

DEFINITION

SQL (Structured Query Language) is the standard declarative language for defining, querying and controlling relational databases.

CATEGORY	COMMANDS
DDL (define)	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML (modify data)	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
DCL (permissions)	`GRANT`, `REVOKE`
TCL (transactions)	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

Joins

JOIN	RETURNS
INNER	Rows with a match in both tables.
LEFT	All left rows + matches from right (NULL if none).
RIGHT	All right rows + matches from left.
FULL	All rows from both, NULLs where no match.
CROSS	Cartesian product of both tables.
SELF	A table joined with itself.

08 Indexing

DEFINITION

An **index** is an auxiliary data structure that speeds up data retrieval at the cost of extra storage and slower writes. Most database indexes are **B+ trees**.

TYPE	NOTES
Primary (clustered)	Data is physically ordered by the key; one per table.
Secondary	Separate structure pointing to data; many per table.
Dense / Sparse	Dense = entry per record; sparse = entry per block.

FORMULAS

B+ tree order (fan-out) p: $p \times P + (p-1) \times K \leq Block$

(P = pointer size, K = key size)

EXAMPLE

Block = 512 B, key = 10 B, pointer = 6 B → $6p + 10(p-1) \leq 512 \rightarrow 16p \leq 522 \rightarrow$ **p = 32**.

09 Transactions & ACID

DEFINITION

A **transaction** is a logical unit of work — a sequence of operations that must be executed as a single, indivisible action (all or nothing).

PROPERTY	MEANING
A — Atomicity	All operations succeed, or none do.
C — Consistency	The DB moves from one valid state to another.
I — Isolation	Concurrent transactions don't interfere.
D — Durability	Committed changes survive crashes.

Transaction states: Active → Partially Committed → Committed (or Failed → Aborted).

10 Concurrency Control

DEFINITION

Concurrency control is the mechanism that manages simultaneous transactions so they don't corrupt shared data, while maximising parallelism.

Lock-based	Shared (S) locks for reads, Exclusive (X) locks for writes.
2-Phase Locking	Growing phase (acquire) then shrinking phase (release). Guarantees serialisability.
Strict 2PL	Holds all locks until commit — avoids cascading rollback.
Timestamp	Orders transactions by time; abort on conflict.
MVCC	Multi-Version Concurrency Control — readers never block writers.

CONFLICTS

Problems concurrency can cause: **dirty read**, **lost update**, **unrepeatable read**, **phantom read**. Isolation levels (Read Uncommitted → Serializable) trade correctness against speed.

11 Deadlocks in DBMS

DEFINITION

A **deadlock** is a situation where two or more transactions wait indefinitely for locks held by each other, so none can proceed.

The four **Coffman conditions** (all must hold): mutual exclusion, hold-and-wait, no preemption, circular wait.

STRATEGY	IDEA
Prevention	Break one condition (e.g. acquire all locks at once).
Avoidance	Use a **wait-for graph** / timestamps (wait-die, wound-wait).
Detection & recovery	Detect a cycle in the wait-for graph, abort a victim.

12 Recovery

DEFINITION

Database recovery restores the database to a consistent state after a failure, using a **log** of all changes.

- **Log-based recovery:** write *before* and *after* values so operations can be undone (rollback) or redone (roll forward).
- **Checkpoints:** periodically flush logs and data so recovery only re-processes the recent part.
- **Shadow paging:** keep a shadow copy of pages; switch atomically on commit.
- **ARIES** — the modern algorithm: analysis, redo, undo.

13 Storage & File Organization

Block	The unit of data transfer between disk and memory.
Heap file	Records stored in no particular order (fast insert, slow search).
Sequential file	Ordered by a key (fast search, slow insert).
Hashed file	Records placed by a hash of the key (fast equality search).

FORMULAS

Records per block = ⌊ block size ÷ record size ⌋

Blocks needed = ⌈ total records ÷ records per block ⌉

14 SQL vs NoSQL

	SQL	NOSQL
Schema	Fixed	Flexible
Scaling	Vertical (mostly)	Horizontal
Consistency	Strong (ACID)	Eventual (BASE)
Types	Relational	Key-value, document, column, graph
Best for	Transactions, joins, correctness	Scale, flexibility, high write

15 Practice Numericals (Solved)

Q1 **R(A,B,C,D) with FDs `AB→C`, `C→D`, `D→A`. Find all candidate keys.**

SOLUTION

- $AB^+ = \{A,B\} \rightarrow C \rightarrow D = \textbf{ABCD}$ ✓ → AB is a key
- $BC^+ = \{B,C\} \rightarrow D \rightarrow A = \textbf{ABCD}$ ✓ → BC is a key
- $BD^+ = \{B,D\} \rightarrow A \rightarrow C = \textbf{ABCD}$ ✓ → BD is a key
- **Candidate keys: AB, BC, BD**

Q2 **R(A,B,C,D) with FDs `AB→C`, `C→D`. Find the highest normal form.**

SOLUTION

- Key = AB (AB^+ = ABCD). Prime = {A,B}; non-prime = {C,D}
- No partial dependency (C depends on the full key) → **2NF holds**
- C → D is non-prime → non-prime (transitive) → **3NF fails**
- **Highest normal form = 2NF**

Q3 **R(A,B,C,D,E) with FDs `A→B`, `B→C`, `C→D`, `D→E`. Find A^+.**

SOLUTION

- $A^+ = \{A\} \rightarrow B \rightarrow C \rightarrow D \rightarrow E$
- **$A^+ = \{A,B,C,D,E\}$ — A is a candidate key**

Q4 **R(A,B,C,D) has a candidate key of 2 attributes. How many superkeys are possible?**

SOLUTION

- Formula: superkeys = $2^{(n-k)}$ where n = 4, k = 2
- $= 2^2$
- **= 4 superkeys**

Q5 **A B+ tree has block size 512 B, key size 10 B, pointer size 6 B. Find the maximum order (fan-out).**

SOLUTION

- Internal node: p pointers + (p−1) keys ≤ block
- $6p + 10(p-1) \leq 512 \rightarrow 16p \leq 522 \rightarrow p \leq 32.6$
- **Max order p = 32**

Q6 **A block is 1024 bytes and a record is 128 bytes. How many records per block, and how many blocks for 10,000 records?**

SOLUTION

- Records/block = $\lfloor 1024 \div 128 \rfloor$ = **8**
- Blocks = $\lceil 10{,}000 \div 8 \rceil$ = **1,250**
- **8 records/block; 1,250 blocks**

Q7 **R(A,B,C,D) with FD `AB→CD` and `C→A`. Is it in BCNF?**

SOLUTION

- Keys: AB^+=ABCD; BC^+=BCAD; so keys = AB, BC
- Check FD C → A: C is not a superkey
- **No — C→A violates BCNF**

Q8 **What is the minimum number of tables needed to represent an M:N relationship between two entities in the relational model?**

SOLUTION

- Each entity → one table, plus a **junction table** for the M:N relationship
- **3 tables (Entity1, Entity2, Relationship)**

Q9 **R(A,B,C) with FD `A→B` , `B→C` . Is the decomposition R1(A,B), R2(B,C) lossless and dependency-preserving?**

SOLUTION

- Common attribute B; B → BC, so B is a key of R2 → **lossless**
- Both FDs preserved (A→B in R1, B→C in R2) → **dependency-preserving**
- **Yes — lossless and dependency-preserving**

16 Quick Revision Cheat-Sheet

- DBMS = software managing a database; beats file systems on redundancy, consistency, concurrency, security.
- 3-schema: External (views) → Conceptual (logical) → Internal (physical).
- Keys: super ⊇ candidate ⊇ primary; foreign key = referential integrity.
- FD X→Y: X uniquely determines Y. Closure X^+ finds keys.
- Normal forms: 1NF (atomic) → 2NF (no partial) → 3NF (no transitive) → BCNF (X is superkey).
- ACID: Atomicity, Consistency, Isolation, Durability.
- Concurrency: 2PL, timestamp, MVCC; problems = dirty read, lost update, phantom read.

- Deadlock: 4 Coffman conditions; prevent / avoid / detect.
- Recovery: log-based (undo/redo), checkpoints, ARIES.
- Index: B+ tree; order p from $p \times P + (p-1) \times K \leq$ block.
- Records/block = ⌊block ÷ record⌋; superkeys = $2^{(n-k)}$.

> MOST-ASKED
>
> **Normalization (find the highest NF)**, **finding candidate keys**, and **ACID** — these come up in almost every DBMS interview.

17 Follow-up Drills

Interviewers rarely stop at the first answer. Each drill below is the main question, then the follow-ups they actually push on — practise the whole chain, not just the first line.

Q Why normalize a database?

↳ **When would you deliberately denormalize?**

When read performance dominates and joins are the bottleneck — for example a reporting table or a feed. You accept duplication and the risk of inconsistency in exchange for fewer joins.

↳ **3NF vs BCNF — what is the practical difference?**

BCNF is stricter: every determinant must be a candidate key. A table can be in 3NF yet still have an anomaly that BCNF removes, usually when there are overlapping candidate keys.

Q Why not just index every column?

↳ **Clustered vs non-clustered index?**

A clustered index defines the physical order of the rows (one per table); a non-clustered index is a separate structure with pointers back to the row. Range scans love clustered indexes.

↳ **When does the optimiser ignore your index?**

When the query returns a large fraction of the table, when a function wraps the indexed column (so it cannot be used), or when statistics say a scan is cheaper.

Q Explain ACID with a concrete example.

↳ **Which isolation level would you pick, and why?**

Read Committed for most OLTP — it blocks dirty reads with low overhead. Serializable only where correctness demands it, because it costs throughput.

↳ **Dirty read vs phantom read?**

A dirty read sees uncommitted data. A phantom read sees new rows that appeared between two identical queries, which row locks alone cannot prevent — you need range or predicate locking.

Q Optimistic vs pessimistic concurrency control?

↳ **How does MVCC change the picture?**

MVCC keeps multiple versions so readers do not block writers and vice versa. You get snapshot isolation cheaply, at the cost of storing old versions and needing cleanup (vacuum).

↳ **How do you handle a write-write conflict under MVCC?**

The database detects that the row version changed since your snapshot and aborts your transaction, so the application must retry.

Q How would you find duplicate rows in a table?

↳ **How do you delete duplicates but keep one?**

Group by the duplicate columns, keep MIN(id) or use ROW_NUMBER() over a partition, then delete the rest — usually inside a transaction so a failure leaves the table intact.

↳ **INNER vs LEFT JOIN — where do NULLs bite?**

INNER JOIN drops non-matching rows; LEFT JOIN keeps them with NULLs. Filtering a LEFT JOIN's right-hand column in WHERE silently turns it into an INNER JOIN — a classic bug.

Q How does a database detect a deadlock?

↳ **Detection vs prevention — which is cheaper?**

Detection (a wait-for graph cycle check) is cheaper at runtime and lets you abort one victim. Prevention (strict ordering, timeouts) avoids the check but can block unnecessarily.

↳ **Which transaction should be the victim?**

Usually the one with the least work done, so the rollback cost is lowest — some engines also consider age to avoid starving old transactions.

Q SQL or NoSQL — how do you decide?

↳ **Which would you choose for a payments ledger?**

A relational database — you need multi-row atomicity, strong consistency and constraints. Money is the last place to trade correctness for scale.

↳ **How do you get transactions in MongoDB?**

Multi-document transactions exist (replica sets required), but the idiomatic approach is to design documents so a single-document atomic update covers the change.

Q How do you read an EXPLAIN plan?

↳ **What does a full table scan sometimes beat an index?**

When you need most of the rows anyway: random I/O through an index becomes more expensive than one sequential scan. The optimiser estimates this from statistics.

↳ **Why do statistics matter?**

They tell the optimiser how selective a predicate is. Stale statistics lead to bad plans — which is why ANALYZE / auto-analyze exists.